DATA STRUCTURES & ALGORITHMS

CASE STUDY REPORT

Balanced Parentheses Checker in a Code Editor

Submitted by

Mohammed Hayat Azad

Rahin Khamkar

Ayaan Surve

Zombarkar Huzaifa

Umair Salmani

Department of Computer Engineering

Mumbai, India

Contents

Abstract **1**

1 Introduction **1**

2 Problem Statement **1**
2.1 Example 2
2.2 Relevance to Code Editors 2

3 Data Structure Used **2**
3.1 What is a Stack? 2
3.2 Key Characteristics 2

4 Why is this Data Structure Suitable? **2**
4.1 Comparison with Alternative Data Structures 3

5 Diagrams **3**
5.1 Stack Evolution Trace 3
5.2 Algorithm Decision-Flow Diagram 3

6 Operations **3**
6.1 How Operations Are Used 4

7 Algorithms **6**
7.1 Pseudocode 6
7.2 Complexity Analysis 6

8 Working of the Case Study **6**
8.1 Integration into a Real Code Editor 7

9 Conclusion **7**

References **9**

Abstract

Modern code editors and Integrated Development Environments (IDEs) such as Visual Studio Code, IntelliJ IDEA, and Sublime Text instantly flag mismatched brackets, braces, and parentheses while a programmer types. This case study examines the **Balanced Parentheses Checker**, a classic application of the **Stack** data structure, and explains how it is used to validate the syntactic correctness of nested symbols in source code. The report covers the problem statement, the choice and justification of the data structure, the operations performed, algorithm design, a fully-worked step-by-step trace with diagrams, a decision-flow diagram of the checking logic, a comparison against alternative data structures, and a discussion of real-world use inside code editors and compilers, followed by a conclusion on efficiency and significance.

---

1 Introduction

Every programming language relies on paired symbols – parentheses `( )`, square brackets `[ ]`, and curly braces { } – to define the structure of expressions, function calls, arrays, and code blocks. A single missing or misplaced bracket can break an entire program, making it fail to compile or run. To help programmers catch such mistakes early, modern code editors continuously check whether every opening symbol has a corresponding, correctly ordered closing symbol. This feature is commonly seen as:

- Bracket highlighting when the cursor is placed near a bracket.
- Red underlines or error markers on unmatched brackets.
- Auto-closing of brackets as the programmer types.
- Auto-indentation based on nested block levels.
- Code folding, which groups a matched pair and everything between them into a collapsible block.

The underlying logic that powers all of these features is the **Balanced Parentheses Problem**, one of the most fundamental and frequently asked problems in Data Structures, solved elegantly using a **Stack**.

2 Problem Statement

Given a string containing only the characters `(`, `)`, `[`, `]`, {, }, determine whether the sequence of brackets is *balanced*, i.e.:

1. Every opening bracket has a matching closing bracket of the *same type*.
2. Brackets are closed in the *correct order* (the most recently opened bracket must be closed first).
3. No closing bracket appears without a corresponding opening bracket.

2.1 Example

Input	Result	Reason
`{[()]}`	Balanced	Every bracket closes in correct nested order
`([)]`	Not Balanced	`)` closes before `[` even though `[` opened later
`((`	Not Balanced	Opening brackets never closed
`))`	Not Balanced	Closing bracket with no matching opener

2.2 Relevance to Code Editors

When a programmer writes code such as:

```
def calculate(a, b):
    result = (a + b) * [1, 2, 3][0]
    return result
```

the editor must verify, character by character, that every `(`, `[`, and their closing counterparts are correctly nested. If they are not, the editor immediately reports a **syntax error**, preventing the programmer from running broken code and saving debugging time.

3 Data Structure Used

The data structure used to solve the balanced parentheses problem is the **Stack**.

3.1 What is a Stack?

A Stack is a linear data structure that follows the **LIFO (Last In, First Out)** principle – the last element inserted is the first one to be removed. It supports two primary operations, `push` (insert) and `pop` (remove), both performed from the same end, called the **top** of the stack.

3.2 Key Characteristics

- Insertion and deletion happen only at the top.
- Follows LIFO ordering.
- Can be implemented using arrays or linked lists.
- All core operations run in **O(1)** constant time.

4 Why is this Data Structure Suitable?

The Stack is the most natural and efficient choice for this problem for the following reasons:

1. **Nested structure matches LIFO behaviour:** Brackets in code are always closed in the reverse order in which they were opened – exactly how a stack behaves. The most recently opened bracket must be the first one closed.

2. **Constant-time operations:** Push and pop both run in O(1), letting the editor validate brackets instantly, even in very large files, without lag.

3. **Simple mismatch detection:** If a closing bracket does not match the type on top of the stack, a mismatch is detected immediately, at the exact character position.

4. **Minimal memory overhead:** Only unmatched opening brackets are stored at any time, unlike other approaches that might require storing the entire string's structure.

5. **Natural end condition:** If the stack is empty at the end of the scan, the string is balanced; if it is non-empty, some opening bracket was left unclosed.

4.1 Comparison with Alternative Data Structures

Structure	Behaviour on this problem	Verdict
Stack	Naturally models "last opened, first closed"; O(1) push/pop	**Ideal**
Queue	FIFO order is the opposite of what nesting requires; would need extra reversal logic	Unsuitable
Plain Array (no stack ops)	Would need manual index tracking to mimic a stack, adding complexity for no benefit	Redundant
Counter (single integer)	Can count depth but cannot distinguish bracket *types*, so mismatches like `([)]` go undetected	Incomplete

No other elementary data structure models "last opened, first closed" as cleanly or efficiently as a stack, which is why it is the industry-standard approach used inside real compilers, linters, and code editors.

5 Diagrams

5.1 Stack Evolution Trace

The diagram below traces the stack's state while scanning the string `{[()]}` from left to right. Each frame shows the character being read, the push/pop action taken, and the resulting stack contents (top of stack is the highlighted frame).

5.2 Algorithm Decision-Flow Diagram

The flowchart below shows the decision logic executed for *every character* read by the editor's lexer.

6 Operations

The stack-based checker relies on four fundamental stack operations:

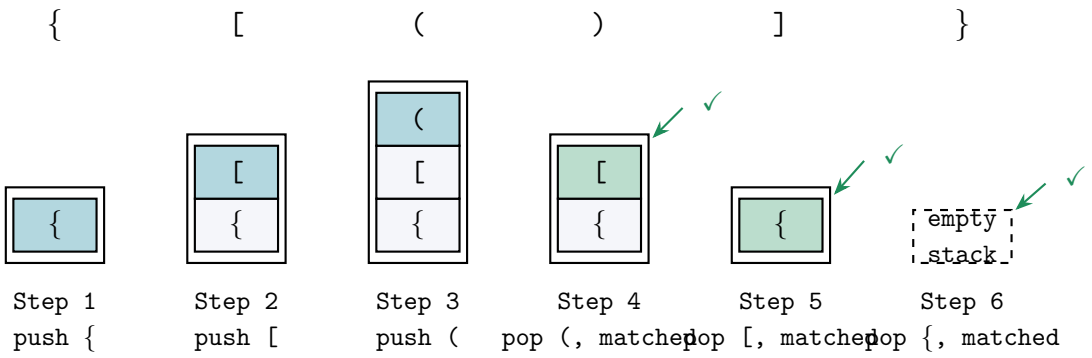


Figure 1: Stack evolution while scanning {[()]}. The final stack is empty ⇒ the string is balanced.

Operation	Description	Time Complexity
`push(x)`	Insert element `x` on top of the stack	O(1)
`pop()`	Remove and return the top element	O(1)
`peek()/top()`	View the top element without removing it	O(1)
`isEmpty()`	Check whether the stack has no elements	O(1)

6.1 How Operations Are Used

- When an **opening** bracket (`(`, `[`, `{`) is encountered, it is **pushed** onto the stack.
- When a **closing** bracket (`)`, `]`, `}`) is encountered, the algorithm calls `peek()` to check the top element and `pop()` to remove it if it matches.
- `isEmpty()` is used both during scanning (to catch an unmatched closing bracket) and at the very end (to confirm no unclosed brackets remain).

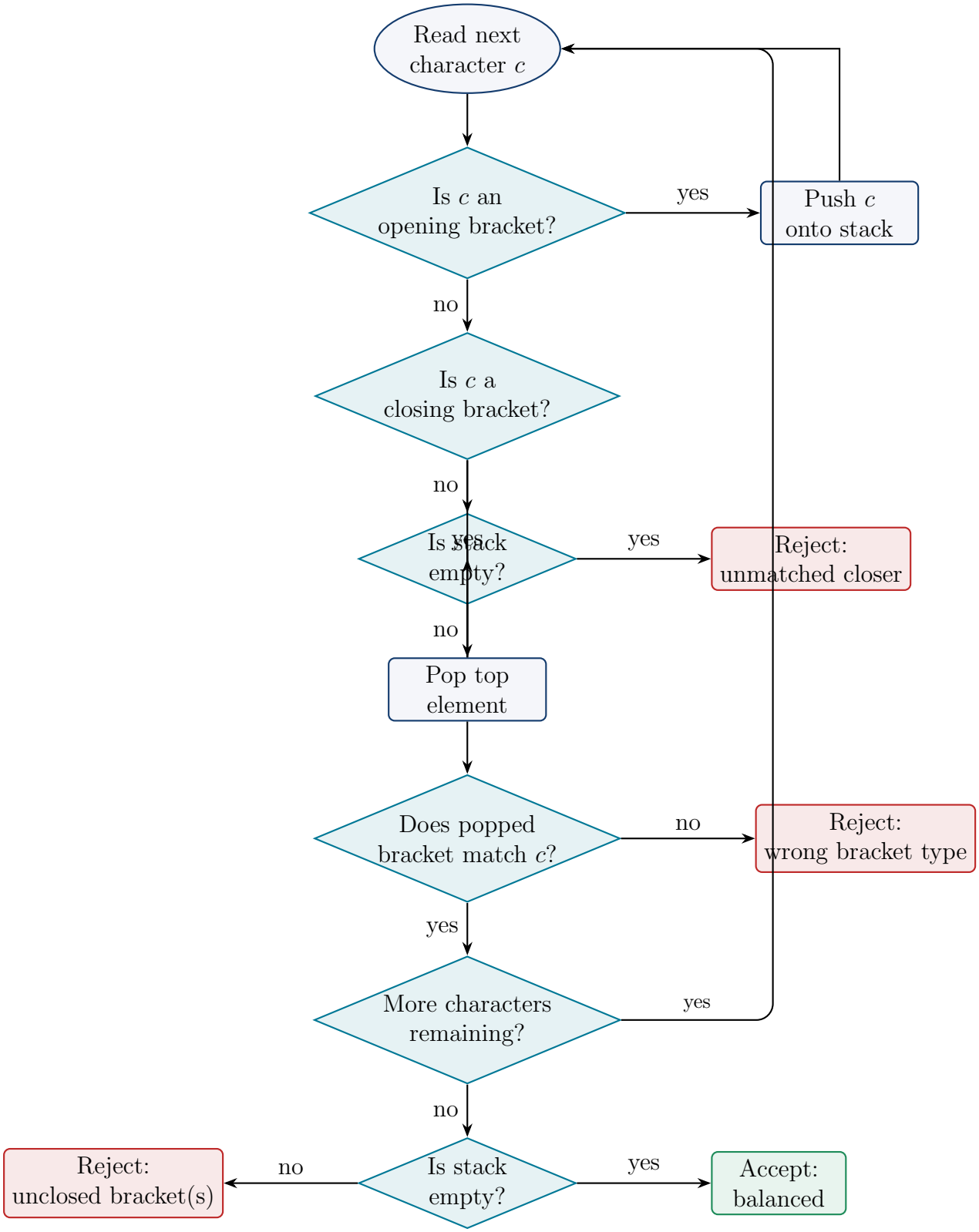


Figure 2: Decision flow used by the checker for each character in the source code.

7 Algorithms

7.1 Pseudocode

Algorithm 1 Balanced Parentheses Checker using Stack

Input: String S of length n containing bracket characters
Output: `true` if balanced, `false` otherwise
Initialize an empty stack St
for each character c in S **do**
    if c is an opening bracket **then**
        $push(St, c)$
    else if c is a closing bracket **then**
        if St is empty **then**
            return `false` ▷ closing bracket with nothing to match
        end if
        $top \leftarrow pop(St)$
        if *top* does not match c **then**
            return `false` ▷ wrong bracket type
        end if
    end if
end for
if St is empty **then**
    return `true`
else
    return `false` ▷ unclosed opening bracket(s) remain
end if

7.2 Complexity Analysis

Metric	Value	Reason
Time Complexity	$O(n)$	Each of the n characters is processed exactly once
Space Complexity	$O(n)$	Worst case: every character is an opening bracket (e.g. `((((`)
Best-case Space	$O(1)$	String has no bracket characters at all

8 Working of the Case Study

Consider the code editor validating the snippet `a[i] = (b + c) * {d - e}`.

Only the bracket characters matter: `[ ] ( ) { }`. The table below shows the stack's state at every step.

Step	Character	Action	Stack (top → right)
1	`[`	push	`[`
2	`]`	pop, matches `[`	*empty*

Step	Character	Action	Stack (top → right)
3	(	push	(
4	)	pop, matches (	*empty*
5	{	push	{
6	}	pop, matches {	*empty*

At the end of the scan, the stack is **empty**, so the editor confirms the code is **syntactically balanced** and allows compilation/execution to proceed.

If, instead, the programmer had typed `a[i) = (b + c)`, the stack would contain `[` when it reaches `)`, causing a mismatch – the editor would immediately underline the `)` in red and report "Expected `]` but found `)`", pointing the programmer to the exact error location.

8.1 Integration into a Real Code Editor

1. The editor's lexer/tokenizer scans the source code character by character as the user types.
2. Every bracket token is passed to the balanced-parentheses checker in real time.
3. On a mismatch, the position (line and column) of the offending character is recorded and passed to the UI layer.
4. The UI layer renders a red squiggly underline or highlight at that position and may show a tooltip with the error message.
5. When the cursor is placed next to a bracket, the same stack logic is used in reverse (matching outward) to highlight its corresponding pair.
6. On save or on-the-fly, the language server can also use the same stack trace to drive auto-indentation and code folding ranges.

9 Conclusion

The Balanced Parentheses Checker is a compact yet powerful demonstration of how a simple data structure – the **Stack** – can solve a problem that is fundamental to every code editor and compiler in use today.

By exploiting the stack's LIFO property, the algorithm matches nested brackets in a single linear pass, achieving an optimal $O(n)$ time complexity with only $O(n)$ auxiliary space.

This case study confirms that:

- Stacks are the ideal structure for any problem involving **nested** or **hierarchical** matching.
- The algorithm is simple to implement, easy to reason about, and highly efficient, making it practical for real-time use in editors handling thousands of lines of code.

- This same underlying logic extends beyond bracket checking to related applications such as **expression evaluation**, **syntax parsing**, **HTML/XML tag validation**, **undo-redo functionality**, and **function call stack tracing** in compilers and interpreters.

In summary, the balanced parentheses checker is a small but essential building block that makes modern code editors faster, smarter, and far more helpful in catching errors before code is even run.

References

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., Stein, C. *Introduction to Algorithms*, MIT Press.

2. Weiss, M. A. *Data Structures and Algorithm Analysis*, Pearson Education.

3. GeeksforGeeks – *Check for Balanced Parentheses in an Expression.*

4. Visual Studio Code Documentation – *Bracket Pair Colorization and Matching.*